

HDOC Day-28

Variable-Length 2D Arrays and Row-Major Order in C

Question 1

Read positive rows and columns, create a variable-length 2D integer array, read elements column by column, and print them in row-major order using one loop and a linear index.

Algorithm - Question 1

Start.

Read rows and repeat until rows > 0.

Read columns and repeat until columns > 0.

Declare `int a[rows][columns]`.

Using nested loops with columns as the outer loop and rows as the inner loop, read `a[i][j]`.

Use one loop `k = 0` to `rows*columns-1` and print the contiguous row-major elements using `*((int *)a + k)`.

Stop.

Test-Case Table - Question 1

Case	Rows	Cols	Input order (column-wise)	Expected row-major output
Valid	2	3	1 4 2 5 3 6	1 2 3 4 5 6
Boundary	1	1	9	9
Invalid then valid	0 -> 2	-1 -> 2	1 3 2 4	1 2 3 4

C Program - Question 1

```
#include <stdio.h>

int main()
{
    int r, c, i, j, k;

    do
    {
        printf("Enter rows: ");
        scanf("%d", &r);
        if (r <= 0)
            printf("Rows must be positive.\n");
    } while (r <= 0);

    do
    {
        printf("Enter columns: ");
        scanf("%d", &c);
        if (c <= 0)
            printf("Columns must be positive.\n");
    } while (c <= 0);

    int a[r][c];

    printf("\nEnter array elements column by column:\n");
    for (j = 0; j < c; j++)
```

```

    {
        for (i = 0; i < r; i++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    printf("\nElements in row-major order:\n");
    for (k = 0; k < r * c; k++)
    {
        printf("%d ", a[k / c][k % c]);
        if ((k + 1) % c == 0)
            printf("\n");
    }

    return 0;
}

```

Case	Rows	Cols	2D input (row-wise)	Expected 1D array
Valid	2	3	1 2 3 4 5 6	1 2 3 4 5 6
Boundary	1	1	7	7
Valid	3	2	1 2 3 4 5 6	1 2 3 4 5 6

```

#include <stdio.h>

int main()
{
    int r, c, i, j, k = 0;

    do
    {
        printf("Enter rows: ");
        scanf("%d", &r);
        if (r <= 0)
            printf("Rows must be positive.\n");
    } while (r <= 0);

    do
    {
        printf("Enter columns: ");
        scanf("%d", &c);
        if (c <= 0)
            printf("Columns must be positive.\n");
    } while (c <= 0);

    int a[r][c];
    int b[r * c];

    printf("\nEnter elements of the 2D array:\n");
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    for (i = 0; i < r; i++)
    {

```

```

        for (j = 0; j < c; j++)
        {
            b[k] = a[i][j];
            k++;
        }
    }

    printf("\n2D array:\n");
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }

    printf("\n1D array in row-major order:\n");
    for (i = 0; i < r * c; i++)
        printf("%d ", b[i]);
    printf("\n");

    return 0;
}

```

Sample Execution

Question 1 sample

```

Enter rows: 2
Enter columns: 3
Enter 6 elements column by column:
1 4 2 5 3 6
Row-major order: 1 2 3 4 5 6

```

Question 2 sample

```

Enter rows: 2
Enter columns: 3
Enter 6 elements row by row:
1 2 3 4 5 6
1D array in row-major order: 1 2 3 4 5 6

```

Screenshot 1 - Question 1 Source Code / Compilation

```

#include <stdio.h>

int main()
{
    int r, c, i, j, k;

    /* Read and validate number of rows */
    do
    {
        printf("Enter rows: ");
        scanf("%d", &r);

        if (r ≤ 0)
            printf("Rows must be positive.\n");
    } while (r ≤ 0);

    /* Read and validate number of columns */
    do
    {
        printf("Enter columns: ");
        scanf("%d", &c);

        if (c ≤ 0)
            printf("Columns must be positive.\n");
    } while (c ≤ 0);

    /* Variable Length 2D Array */
    int a[r][c];

    printf("\nEnter array elements column by column:\n");

```

```

    /* Input column by column */
    for (j = 0; j < c; j++)
    {
        for (i = 0; i < r; i++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    printf("\nElements in row-major order:\n");

    /*
    Print using a single loop and linear index.
    k / c gives the row index.
    k % c gives the column index.
    */
    for (k = 0; k < r * c; k++)
    {
        printf("%d ", a[k / c][k % c]);

        if ((k + 1) % c == 0)
            printf("\n");
    }

    return 0;
}

```

```

(kali@kali)-[~/Desktop]
$ vi hdoc28.c

(kali@kali)-[~/Desktop]
$ gcc hdoc28.c -o hdoc28 -lm

```

Screenshot 2 - Question 1 Output

Test Case 1:

```
(kali㉿kali)-[~/Desktop]
$ ./hdoc28
Enter rows: 2
Enter columns: 3

Enter array elements column by column:
a[0][0] = 1 4 2 5 3 6
a[1][0] = a[0][1] = a[1][1] = a[0][2] = a[1][2] =
Elements in row-major order:
1 2 3
4 5 6
```

Test Case 2:

```
(kali㉿kali)-[~/Desktop]
$ ./hdoc28
Enter rows: 1
Enter columns: 1

Enter array elements column by column:
a[0][0] = 9

Elements in row-major order:
9
```

Test Case 3:

```
(kali㉿kali)-[~/Desktop]
$ ./hdoc28
Enter rows: 0
Rows must be positive.
Enter rows: 2
Enter columns: -1
Columns must be positive.
Enter columns: 2

Enter array elements column by column:
a[0][0] = 1 3 2 4
a[1][0] = a[0][1] = a[1][1] =
Elements in row-major order:
1 2
3 4
```

Screenshot 3 - Question 2 Source Code / Compilation

```

#include <stdio.h>

int main()
{
    int r, c, i, j, k = 0;

    /* Read and validate number of rows */
    do
    {
        printf("Enter rows: ");
        scanf("%d", &r);

        if (r ≤ 0)
            printf("Rows must be positive.\n");
    } while (r ≤ 0);

    /* Read and validate number of columns */
    do
    {
        printf("Enter columns: ");
        scanf("%d", &c);

        if (c ≤ 0)
            printf("Columns must be positive.\n");
    } while (c ≤ 0);

    /* Variable Length Arrays */
    int a[r][c];
    int b[r * c];

    printf("\nEnter elements of the 2D array:\n");

```

```

    /* Read elements into 2D array */
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* Copy 2D array into 1D array in row-major order */
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            b[k] = a[i][j];
            k++;
        }
    }

    printf("\n2D array:\n");

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

```

```

    printf("\n");

    printf("\n1D array in row-major order:\n");

    for (i = 0; i < r * c; i++)
    {
        printf("%d ", b[i]);
    }

    printf("\n");

    return 0;
}

```

```

(kali㉿kali)-[~/Desktop]
$ vi hdoc28_q2.c

(kali㉿kali)-[~/Desktop]
$ gcc hdoc28_q2.c -o hdoc28_q2 -lm

```

Screenshot 4 - Question 2 Output

Test Case 1:

```

(kali㉿kali)-[~/Desktop]
$ ./hdoc28_q2
Enter rows: 2
Enter columns: 3

Enter elements of the 2D array:
a[0][0] = 1 2 3 4 5 6
a[0][1] = a[0][2] = a[1][0] = a[1][1] = a[1][2] =
2D array:
1 2 3
4 5 6

1D array in row-major order:
1 2 3 4 5 6

```

Test Case 2:

```

(kali㉿kali)-[~/Desktop]
$ ./hdoc28_q2
Enter rows: 1
Enter columns: 1

Enter elements of the 2D array:
a[0][0] = 7

2D array:
7

1D array in row-major order:
7

```

Test Case 3:

```

(kali㉿kali)-[~/Desktop]
$ ./hdoc28_q2
Enter rows: 3
Enter columns: 2

Enter elements of the 2D array:
a[0][0] = 1 2 3 4 5 6
a[0][1] = a[1][0] = a[1][1] = a[2][0] = a[2][1] =
2D array:
1 2
3 4
5 6

1D array in row-major order:
1 2 3 4 5 6

```